ICPC Code Template

喵喵喵幼儿园

4 CATEGORIES **5** TEMPLATES **96** SOURCE LINES

Quick Index

1 Data Structures

1.1 Disjoint Set Union

FILE data-structures/disjoint_set_union.cpp **LINES** 28

USE Maintain components with path compression and union by size. **COST** O(alpha(n)) amortized per operation

AUTHOR Team Template

```cpp
struct DSU {
    std::vector<int> parent, size;

    explicit DSU(int n) : parent(n), size(n, 1) {
        std::iota(parent.begin(), parent.end(), 0);
    }

    int find(int x) {
        while (x != parent[x]) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }

    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (size[a] < size[b]) std::swap(a, b);
        parent[b] = a;
        size[a] += size[b];
        return true;
    }

    bool same(int a, int b) { return find(a) == find(b); }
    int component_size(int x) { return size[find(x)]; }
};
```

2 Graph

2.1 Dijkstra Shortest Paths

FILE graph/dijkstra.cpp **LINES** 30

USE Single-source shortest paths in a graph with non-negative edge weights. **COST** O((n + m) log n)

AUTHOR Team Template

```cpp
using i64 = long long;
constexpr i64 INF = std::numeric_limits<i64>::max() / 4;

struct Edge {
    int to;
    i64 weight;
};

std::vector<i64> dijkstra(const std::vector<std::vector<Edge>>& graph, int source) {
    const int n = static_cast<int>(graph.size());
    std::vector<i64> distance(n, INF);
    using State = std::pair<i64, int>;
    std::priority_queue<State, std::vector<State>, std::greater<State>> queue;

    distance[source] = 0;
    queue.emplace(0, source);
    while (!queue.empty()) {
        auto [current_distance, vertex] = queue.top();
        queue.pop();
        if (current_distance != distance[vertex]) continue;

        for (const Edge& edge : graph[vertex]) {
            const i64 candidate = current_distance + edge.weight;
            if (candidate >= distance[edge.to]) continue;
            distance[edge.to] = candidate;
            queue.emplace(candidate, edge.to);
        }
    }
    return distance;
}
```

3 Math

3.1 Binary GCD

FILE math/binary_gcd.cpp **LINES** 6

USE 超快 gcd 算法 **COST** O(log n)，可以当作 O(1)

AUTHOR ppip

```cpp
#define ctz __builtin_ctz
inline int gcd(int a,int b) {
    int az{ctz(a)},bz{ctz(b)},z{min(az,bz)},t;b>>=bz;
    while (a) a>>=az,t=a-b,az=ctz(t),b=min(a,b),a=abs(t);
    return b<<z;
}
```

3.2 Modular Power and Inverse

FILE math/modular_power.cpp **LINES** 16

USE Binary exponentiation and multiplicative inverse modulo a prime. **COST** O(log exponent)

AUTHOR Team Template

```cpp
using i64 = long long;

i64 modular_power(i64 base, i64 exponent, i64 modulus) {
    base %= modulus;
    i64 result = 1 % modulus;
    while (exponent > 0) {
        if (exponent & 1) result = static_cast<__int128>(result) * base % modulus;
        base = static_cast<__int128>(base) * base % modulus;
        exponent >>= 1;
    }
    return result;
}

i64 modular_inverse_prime(i64 value, i64 prime_modulus) {
    return modular_power(value, prime_modulus - 2, prime_modulus);
}
```

4 Misc

4.1 快速模乘

FILE misc/fastmod.cpp **LINES** 16

USE 要求 1<=P<2^31 且 a<P；同一 z 多次使用时先 trans，再用 mul_t **COST** O(1)

AUTHOR ppip

```cpp
using u32=unsigned;
using u64=unsigned long long;
using u128=unsigned __int128;
const u32 P=998244353;
inline u64 trans(u64 x) {
    constexpr u64 A=-(u64)P/P+1;
    constexpr u64 q=((u128(-(u64)P%P)<<64)+P-1)/P;
    return x*A+u64((u128)x*q>>64)+1;
}
// t 必须是 trans(z) 的返回值；复用同一 z 时只需计算一次 t
inline u32 mul_t(u32 a,u64 t) {
    return a*t*(u128)P>>64;
}
inline u32 mul(u32 a,u64 z) {
    return mul_t(a,trans(z));
}
```